Report_ekstraklasa_analysis_24_25

March 12, 2026

1 Introduction

Celem niniejszego raportu jest przeprowadzenie kompleksowej analizy statystycznej drużyn Ekstraklasy w sezonie 2024/2025 w oparciu o dane pochodzące z serwisu FBref.com. W analizie wykorzystano zarówno podstawowe statystyki meczowe (gole zdobyte i stracone), jak i wskaźniki stylu gry oraz efektywności ofensywnej, które po oczyszczeniu i scaleniu posłużyły do dalszych badań. Kluczową częścią raportu jest analiza eksploracyjna (EDA), która poprzez wykresy i analizę korelacji pokazuje, że największy wpływ na liczbę punktów mają jakość strzałów i skuteczność wykończenia, a nie samo posiadanie piłki. Porównanie drużyn z czołówki, środka i dołu tabeli wskazuje, że zespoły z TOP4 wyróżniają się większą liczbą celnych strzałów i lepszym bilansem bramkowym, podczas gdy drużyny z dolnej części tabeli mają problemy z efektywnością ofensywną i defensywą. Zastosowany model regresji liniowej osiągnął wysokie R^2, co potwierdza, że statystyki ofensywne i defensywne dobrze wyjaśniają zdobycz punktową, choć ze względu na ograniczoną próbę analiza ma charakter głównie opisowy.

2 Data cleaning

[47]:
```
import pandas as pd

# Wczytanie konkretnego arkusza
df_table = pd.read_excel("../data/Dane_Ekstraklasa.xlsx",
                         sheet_name="Tabela_ligi_24_25",
                         engine="openpyxl")


df_table.head()
```

[47]:
```
   Rk           Squad  MP   W   D   L  GF  GA  GD  Pts  Pts/MP  Attendance  \
0   1     Lech Poznań  34  22   4   8  68  31  37   70    2.06      28.947
1   2           Raków  34  20   9   5  51  23  28   69    2.03       5.427
2   3     Jagiellonia  34  17  10   7  56  42  14   61    1.79      16.475
3   4  Pogoń Szczecin  34  17   7  10  59  40  19   58    1.71      17.792
4   5    Legia Warsaw  34  15   9  10  60  45  15   54    1.59      24.691

                    Top Team Scorer           Goalkeeper  \
0                Mikael Ishak - 21       Bartosz Mrozek
1        Jonatan Braut Brunes - 14     Kacper Trelowski
```

```
2                  Jesús Imaz - 16  Sławomir Abramowicz
3          Efthimis Koulouris - 28    Valentin Cojocaru
4  Marc Gual, Bartosz Kapustka - 9       Kacper Tobiasz

                                   Notes
0   → Champions League via league finish
1  → Conference League via league finish
2  → Conference League via league finish
3                                    NaN
4            → Europa League via cup win
```

2.0.1 Podstawowe czyszczenie

[48]:
```
df_table = df_table[df_table['Squad'].notna()].copy()
```

W kolumnie 'Squad' odfiltrowywuje NaN albo „Total"

[49]:
```
# Poprawienie nazw drużyn
df_table['Squad'] = df_table['Squad'].str.extract(r'\[(.*?)\]', expand=False).
 ↪fillna(df_table['Squad'])
df_table['Squad'] = df_table['Squad'].str.strip()

# Rzutowanie kolumn na typ numeryczny:
num_cols = ['Rk', 'MP', 'W', 'D', 'L', 'GF', 'GA', 'GD', 'Pts', 'Pts/MP',␣
 ↪'Attendance']
for col in num_cols:
    df_table[col] = pd.to_numeric(df_table[col], errors='coerce')
```

2.0.2 Sprawdzenie braków danych:

[50]:
```
df_table.isna().sum()
```

[50]:
```
Rk                 0
Squad              0
MP                 0
W                  0
D                  0
L                  0
GF                 0
GA                 0
GD                 0
Pts                0
Pts/MP             0
Attendance         0
Top Team Scorer    0
Goalkeeper         0
Notes             11
```

```
dtype: int64
```

2.1 Połączenie kilku akruszy

[51]:
```
# wczytanie dwóch arkuszy
std = pd.read_excel("../data/Dane_Ekstraklasa.xlsx",
                    sheet_name="Squad Standard Stats",
                    engine="openpyxl",
                    skiprows=1)

shoot = pd.read_excel("../data/Dane_Ekstraklasa.xlsx",
                      sheet_name="Squad Shooting",
                      engine="openpyxl",
                      skiprows=1)

# kolumna 'Squad' ma taką samą czystą postać jak w df_table
for df in [std, shoot]:
    df['Squad'] = df['Squad'].str.extract(r'\[(.*?)\]', expand=False).
 ↪fillna(df['Squad'])
    df['Squad'] = df['Squad'].str.strip()

# tylko interesujące nas kolumny
std_small = std[['Squad', '# Pl', 'Age', 'Poss', 'Gls', 'Ast', 'G+A']].rename(
    columns={'Gls': 'Gls_std', 'Ast': 'Ast_std', 'G+A': 'G+A_std'}
)
shoot_small = shoot[['Squad', 'Gls', 'Sh', 'SoT', 'Sh/90', 'SoT/90', 'G/Sh', 'G/
 ↪SoT']].rename(
    columns={'Gls': 'Gls_shoot'}
)

# Merge po nazwie drużyny
df = df_table.merge(std_small, on='Squad', how='left', suffixes=('', '_std'))
df = df.merge(shoot_small, on='Squad', how='left', suffixes=('', '_shoot'))

df.head()
```

[51]:
```
   Rk           Squad  MP   W   D   L  GF  GA  GD  Pts  …  Gls_std  Ast_std  \
0   1     Lech Poznań  34  22   4   8  68  31  37   70  …       67       49
1   2           Raków  34  20   9   5  51  23  28   69  …       50       37
2   3     Jagiellonia  34  17  10   7  56  42  14   61  …       55       40
3   4  Pogoń Szczecin  34  17   7  10  59  40  19   58  …       56       38
4   5    Legia Warsaw  34  15   9  10  60  45  15   54  …       59       39

  G+A_std Gls_shoot   Sh  SoT  Sh/90  SoT/90  G/Sh  G/SoT
0     116        67  531  183  15.62    5.38  0.12   0.34
1      87        50  469  164  13.79    4.82  0.10   0.28
2      95        55  480  152  14.12    4.47  0.10   0.33
```

```
3      94        56  513  180  15.09    5.29  0.10   0.28
4      98        59  520  172  15.29    5.06  0.10   0.30

[5 rows x 28 columns]
```

[52]:
```
df.shape
```

[52]:
```
(18, 28)
```

tabela posiada 18 wierszy i 28 kolumn

[53]:
```
df['Squad'].unique()
```

[53]:
```
array(['Lech Poznań', 'Raków', 'Jagiellonia', 'Pogoń Szczecin',
       'Legia Warsaw', 'Cracovia', 'Motor Lublin', 'Katowice',
       'Górnik Zabrze', 'Piast Gliwice', 'Korona Kielce',
       'Radomiak Radom', 'Widzew Łódź', 'Lechia Gdańsk', 'Zagłębie Lubin',
       'Stal Mielec', 'Śląsk Wrocław', 'Puszcza Niepołomice'],
      dtype=object)
```

[54]:
```
df.isna().sum()
```

[54]:
```
Rk                 0
Squad              0
MP                 0
W                  0
D                  0
L                  0
GF                 0
GA                 0
GD                 0
Pts                0
Pts/MP             0
Attendance         0
Top Team Scorer    0
Goalkeeper         0
Notes             11
# Pl               0
Age                0
Poss               0
Gls_std            0
Ast_std            0
G+A_std            0
Gls_shoot          0
Sh                 0
SoT                 0
Sh/90              0
SoT/90             0
```

```
G/Sh                0
G/SoT               0
dtype: int64
```

brak pustych wartości

2.1.1 Zapis wyczyszczonych danych

[55]:
```python
df.to_csv(
    "../data/ekstraklasa_clean.csv",
    index=False,
    encoding='utf-8'
)
```

3 EDA

[57]:
```python
df = pd.read_csv("../data/ekstraklasa_clean.csv")
df.head()
```

[57]:
```
   Rk           Squad  MP   W   D   L  GF  GA  GD  Pts  …  Gls_std  Ast_std  \
0   1     Lech Poznań  34  22   4   8  68  31  37   70  …       67       49
1   2           Raków  34  20   9   5  51  23  28   69  …       50       37
2   3     Jagiellonia  34  17  10   7  56  42  14   61  …       55       40
3   4  Pogoń Szczecin  34  17   7  10  59  40  19   58  …       56       38
4   5    Legia Warsaw  34  15   9  10  60  45  15   54  …       59       39

  G+A_std Gls_shoot   Sh  SoT  Sh/90  SoT/90  G/Sh  G/SoT
0     116        67  531  183  15.62    5.38  0.12   0.34
1      87        50  469  164  13.79    4.82  0.10   0.28
2      95        55  480  152  14.12    4.47  0.10   0.33
3      94        56  513  180  15.09    5.29  0.10   0.28
4      98        59  520  172  15.29    5.06  0.10   0.30

[5 rows x 28 columns]
```

3.1 Podstawowa eksploracja danych

[58]:
```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 18 entries, 0 to 17
Data columns (total 28 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Rk               18 non-null     int64
 1   Squad            18 non-null     object
 2   MP               18 non-null     int64
 3   W                18 non-null     int64
```

```
 4   D                18 non-null     int64
 5   L                18 non-null     int64
 6   GF               18 non-null     int64
 7   GA               18 non-null     int64
 8   GD               18 non-null     int64
 9   Pts              18 non-null     int64
 10  Pts/MP           18 non-null     float64
 11  Attendance       18 non-null     float64
 12  Top Team Scorer  18 non-null     object
 13  Goalkeeper       18 non-null     object
 14  Notes            7 non-null      object
 15  # Pl             18 non-null     int64
 16  Age              18 non-null     object
 17  Poss             18 non-null     float64
 18  Gls_std          18 non-null     int64
 19  Ast_std          18 non-null     int64
 20  G+A_std          18 non-null     int64
 21  Gls_shoot        18 non-null     int64
 22  Sh               18 non-null     int64
 23  SoT              18 non-null     int64
 24  Sh/90            18 non-null     float64
 25  SoT/90           18 non-null     float64
 26  G/Sh             18 non-null     float64
 27  G/SoT            18 non-null     float64
dtypes: float64(7), int64(16), object(5)
memory usage: 4.1+ KB
```

[59]:
```
df.describe().T
```

[59]:

	count	mean	std	min	25%	50%	\
Rk	18.0	9.500000	5.338539	1.000	5.25000	9.5000	
MP	18.0	34.000000	0.000000	34.000	34.00000	34.0000	
W	18.0	12.722222	4.443178	6.000	10.25000	12.0000	
D	18.0	8.555556	2.202197	4.000	7.00000	8.5000	
L	18.0	12.722222	3.892762	5.000	10.25000	13.0000	
GF	18.0	46.833333	10.071801	33.000	38.00000	46.0000	
GA	18.0	46.833333	10.405033	23.000	40.50000	48.0000	
GD	18.0	0.000000	17.221737	-26.000	-14.00000	-1.5000	
Pts	18.0	46.722222	12.418229	28.000	37.75000	46.0000	
Pts/MP	18.0	1.373889	0.365741	0.820	1.11250	1.3500	
Attendance	18.0	12.698889	7.262120	2.159	6.11975	11.6985	
# Pl	18.0	30.666667	3.253957	24.000	30.00000	30.5000	
Poss	18.0	50.055556	4.514841	40.000	47.72500	48.6500	
Gls_std	18.0	45.444444	9.817285	33.000	36.25000	44.5000	
Ast_std	18.0	31.333333	8.478346	19.000	25.00000	33.0000	
G+A_std	18.0	76.777778	18.050764	54.000	61.00000	78.5000	
Gls_shoot	18.0	45.444444	9.817285	33.000	36.25000	44.5000	

```
Sh           18.0  471.166667  38.562783  374.000  452.25000  473.5000
SoT          18.0  149.500000  17.981199  117.000  142.00000  144.5000
Sh/90        18.0   13.857222   1.134164   11.000   13.29750   13.9250
SoT/90       18.0    4.397222   0.527601    3.440    4.18000    4.2500
G/Sh         18.0    0.087222   0.015645    0.060    0.07250    0.0900
G/SoT        18.0    0.272222   0.042917    0.200    0.24250    0.2700

                 75%      max
Rk           13.7500   18.000
MP           34.0000   34.000
W            14.7500   22.000
D            10.0000   12.000
L            16.0000   18.000
GF           54.7500   68.000
GA           53.0000   63.000
GD           11.7500   37.000
Pts          53.2500   70.000
Pts/MP        1.5675    2.060
Attendance   17.0530   28.947
# Pl         32.5000   36.000
Poss         53.5000   58.800
Gls_std      53.7500   67.000
Ast_std      37.7500   49.000
G+A_std      91.5000  116.000
Gls_shoot    53.7500   67.000
Sh          497.2500  531.000
SoT         162.2500  183.000
Sh/90        14.6275   15.620
SoT/90        4.7700    5.380
G/Sh          0.1000    0.120
G/SoT         0.2975    0.340
```

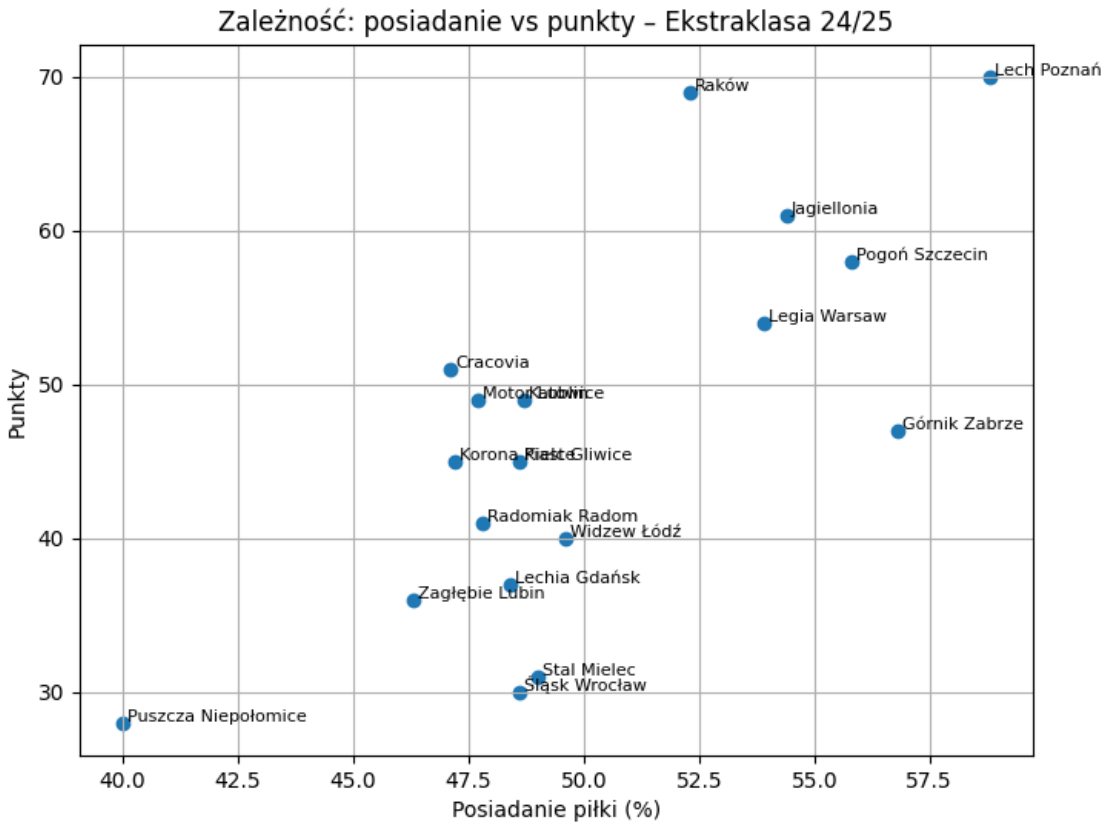
Statystyka opisowa dla zmiennej Poss (posiadanie piłki):

Średnie posiadanie piłki wynosi ok. 50,06%. Odchylenie standardowe mówi że przeciętnie posiadanie piłki odbiegało od średniej o około 4,5 punkta procentowego. Najmniejsze posiadanie piłki wynosiło 40%, a największe 58%. Kwartyle mówią nam że 25% pierwszych obserwacji ma posiadanie mniejsze bądź równe 47,73%, połowa obserwacji ma posiadanie mniejsze niż 48,65%, a połowa wyższe, natomiast 75% pierwszych obserwacji ma posiadanie mniejsze bądź równe 53,5%.

3.1.1 Wykres rozrzutu: Pts vs Poss

[66]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))
plt.scatter(df['Poss'], df['Pts'])

for _, row in df.iterrows():
```

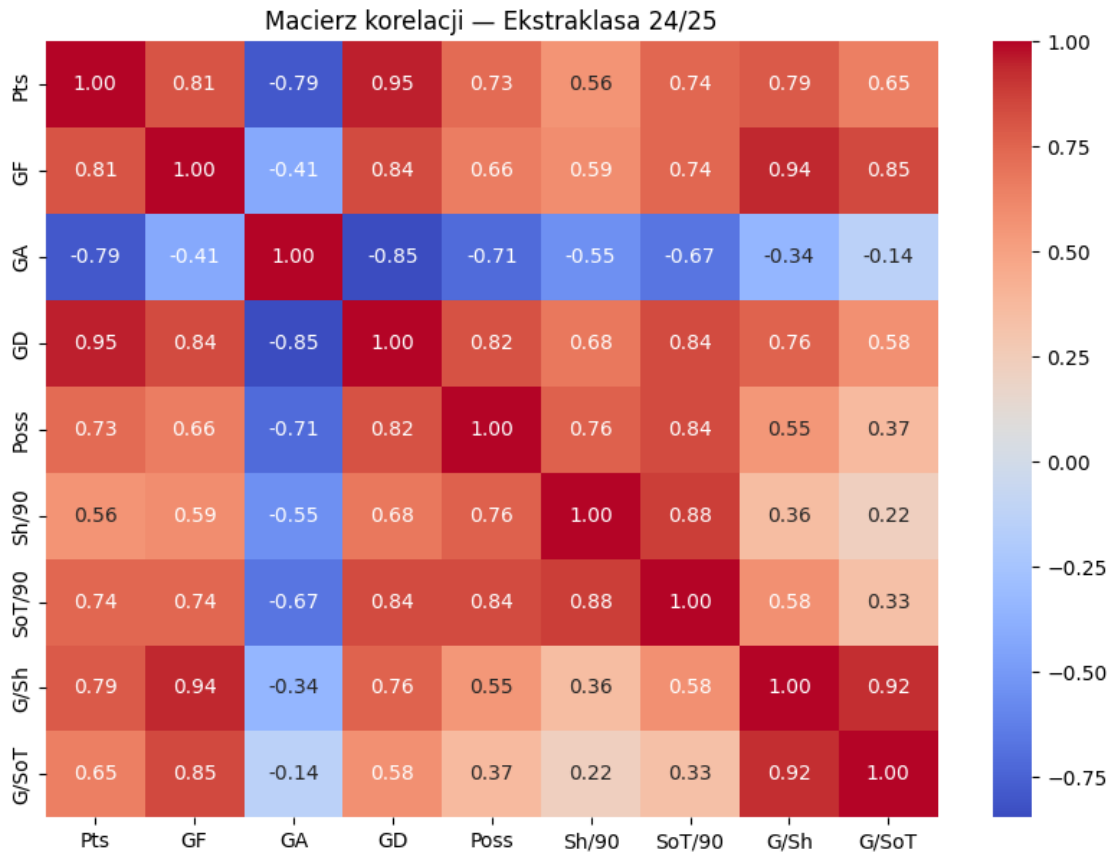
```
    plt.text(row['Poss'] + 0.1, row['Pts'] + 0.1, row['Squad'], fontsize=8)

plt.xlabel("Posiadanie piłki (%)")
plt.ylabel("Punkty")
plt.title("Zależność: posiadanie vs punkty - Ekstraklasa 24/25")
plt.grid(True)
plt.show()
```



Wykres rozrzutu pokazał, że choć drużyny z górnej części tabeli częściej utrzymywały się przy piłce, zależność ta nie jest silnie deterministyczna. Lech i Raków, znajdujące się w prawym-górnym obszarze wykresu, łączą wysokie posiadanie piłki z dużą liczbą punktów, jednak Górnik mimo jednego z najwyższych poziomów posiadania plasuje się znacznie niżej. Z drugiej strony Puszcza, utrzymująca się przy piłce najrzadziej, zdobyła zdecydowanie najmniej punktów. Wskazuje to na to, że posiadanie piłki może pełnić funkcję wspierającą, lecz nie jest kluczowym czynnikiem decydującym o wynikach.

3.2 Macierz korelacji

[67]:
```
import seaborn as sns
import matplotlib.pyplot as plt

corr = df[['Pts','GF','GA','GD','Poss','Sh/90','SoT/90','G/Sh','G/SoT']].corr()

plt.figure(figsize=(10,7))
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f')
plt.title("Macierz korelacji - Ekstraklasa 24/25")
plt.show()
```



[70]:
```
corr['Pts'].sort_values(ascending=False)
```

[70]:
```
Pts       1.000000
GD        0.952504
GF        0.809482
G/Sh      0.789049
SoT/90    0.742636
Poss      0.732407
```

```
G/SoT     0.648018
Sh/90     0.564860
GA       -0.792966
Name: Pts, dtype: float64
```

Macierz korelacji ujawnia, że najsilniejszym „statystycznym wyjaśnieniem” liczby punktów jest różnica bramek (GD = 0,95), liczba goli strzelonych (GF = 0,81) oraz wskaźniki skuteczności strzałów (G/Sh = 0,79 oraz G/SoT = 0,65). Z kolei posiadanie piłki, choć często podkreślane w analizach medialnych, wykazuje mniejszą korelację (Poss = 0,73). W praktyce oznacza to, że drużyny nie wygrywają meczów poprzez długie utrzymywanie się przy piłce, lecz dzięki umiejętności tworzenia i wykorzystywania klarownych sytuacji strzeleckich.

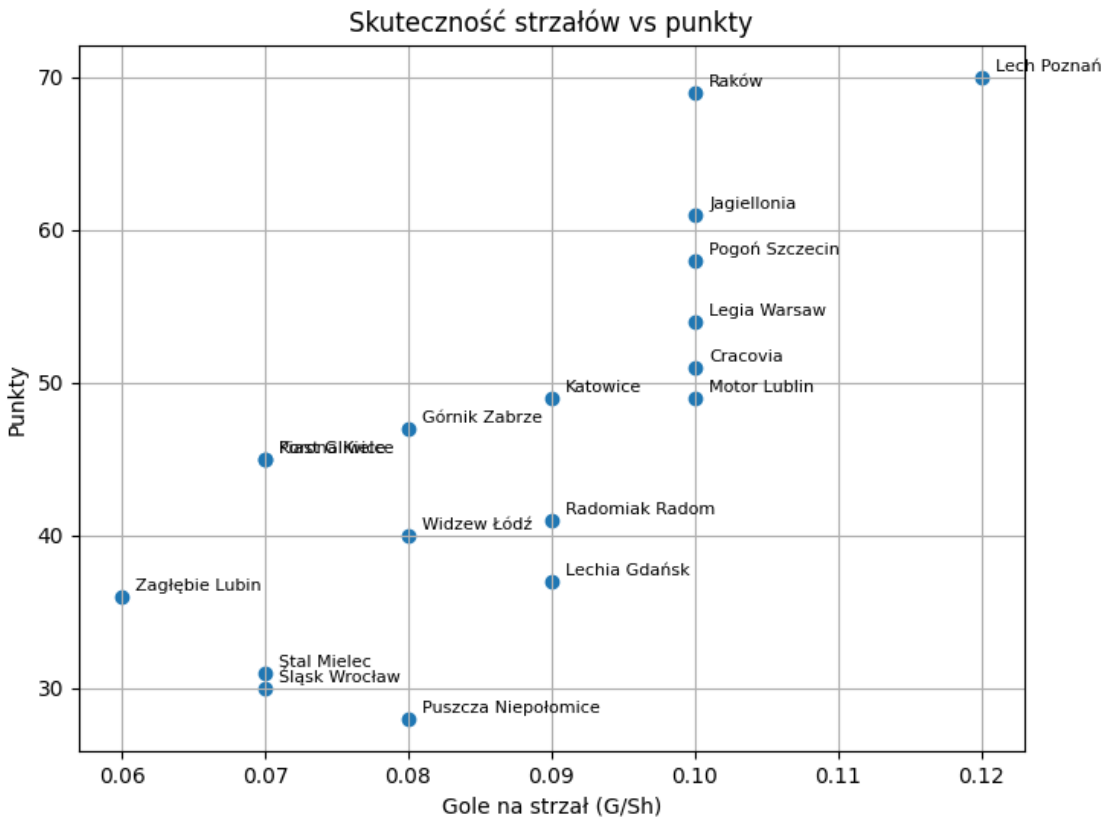
3.3 Skuteczność strzałów

[71]:
```
df[['Squad','G/Sh','G/SoT']].sort_values(by='G/Sh', ascending=False)
```

[71]:

	Squad	G/Sh	G/SoT
0	Lech Poznań	0.12	0.34
1	Raków	0.10	0.28
2	Jagiellonia	0.10	0.33
3	Pogoń Szczecin	0.10	0.28
4	Legia Warsaw	0.10	0.30
5	Cracovia	0.10	0.34
6	Motor Lublin	0.10	0.32
7	Katowice	0.09	0.27
13	Lechia Gdańsk	0.09	0.27
11	Radomiak Radom	0.09	0.29
12	Widzew Łódź	0.08	0.27
8	Górnik Zabrze	0.08	0.25
17	Puszcza Niepołomice	0.08	0.26
9	Piast Gliwice	0.07	0.22
10	Korona Kielce	0.07	0.23
15	Stal Mielec	0.07	0.21
16	Śląsk Wrocław	0.07	0.24
14	Zagłębie Lubin	0.06	0.20

[72]:
```
plt.figure(figsize=(8,6))
plt.scatter(df['G/Sh'], df['Pts'])

for _,row in df.iterrows():
    plt.text(row['G/Sh']+0.001, row['Pts']+0.4, row['Squad'], fontsize=8)

plt.xlabel("Gole na strzał (G/Sh)")
plt.ylabel("Punkty")
plt.title("Skuteczność strzałów vs punkty")
plt.grid(True)
plt.show()
```



Czołówka tabeli przeważa w skuteczności oddawanych strzałów, co faktycznie pokazuje ich znaczenie. Drużyny z końca tabeli zajmują raczej końcowe miejsca w rankingu skuteczności strzałów.

3.4 Ranking drużyn w kluczowych metrykach

3.4.1 Ranking ofensywy (gole strzelone)

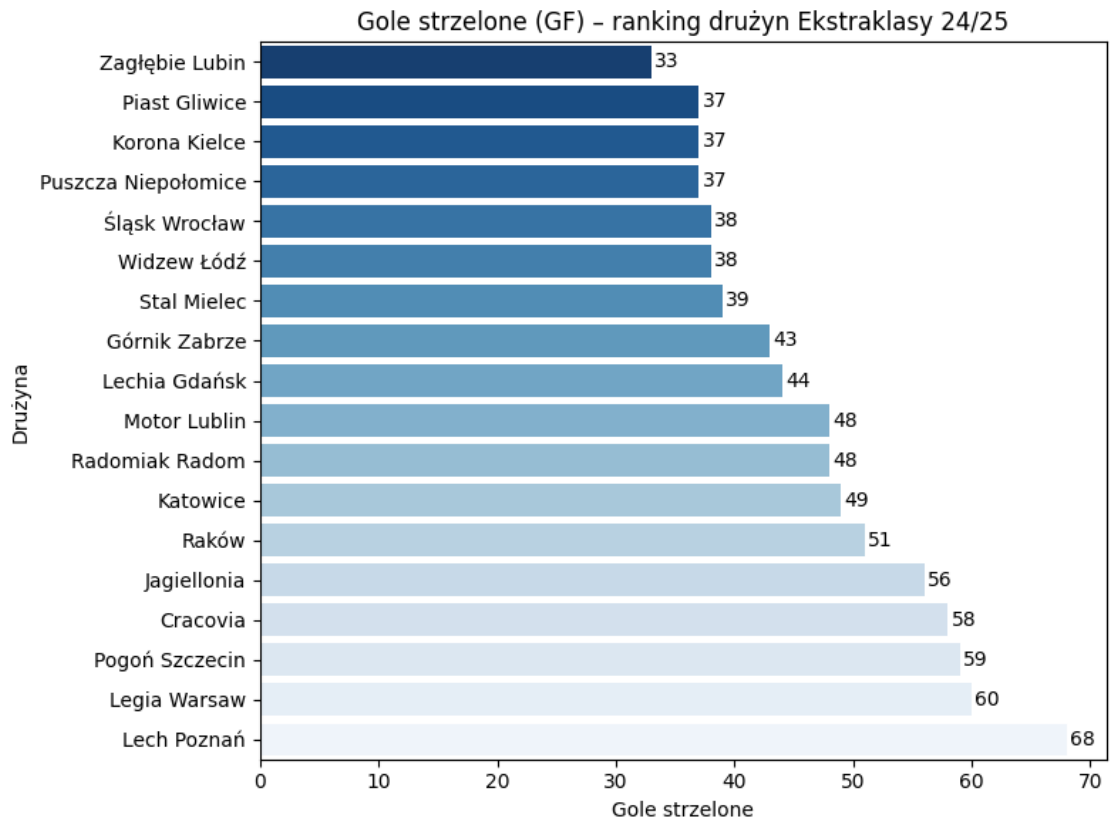
[73]:
```
df[['Squad','GF']].sort_values('GF', ascending=False)
```

[73]:

	Squad	GF
0	Lech Poznań	68
4	Legia Warsaw	60
3	Pogoń Szczecin	59
5	Cracovia	58
2	Jagiellonia	56
1	Raków	51
7	Katowice	49
6	Motor Lublin	48
11	Radomiak Radom	48
13	Lechia Gdańsk	44
8	Górnik Zabrze	43

```
15          Stal Mielec  39
16        Śląsk Wrocław  38
12          Widzew Łódź  38
9         Piast Gliwice  37
10        Korona Kielce  37
17  Puszcza Niepołomice  37
14       Zagłębie Lubin  33
```

[87]:
```python
df_offense = df[['Squad', 'GF']].sort_values('GF', ascending=True)  #␣
↪ascending=True, bo zrobimy poziomy wykres

plt.figure(figsize=(8,6))
sns.barplot(
    data=df_offense,
    x='GF',
    y='Squad',
    hue='Squad',
    palette='Blues_r',
    legend=False
)

plt.title("Gole strzelone (GF) - ranking drużyn Ekstraklasy 24/25")
plt.xlabel("Gole strzelone")
plt.ylabel("Drużyna")

# dodanie wartości na słupkach
for i, v in enumerate(df_offense['GF']):
    plt.text(v + 0.3, i, str(v), va='center')

plt.tight_layout()
plt.show()
```



3.4.2 Ranking defensywy (gole stracone)

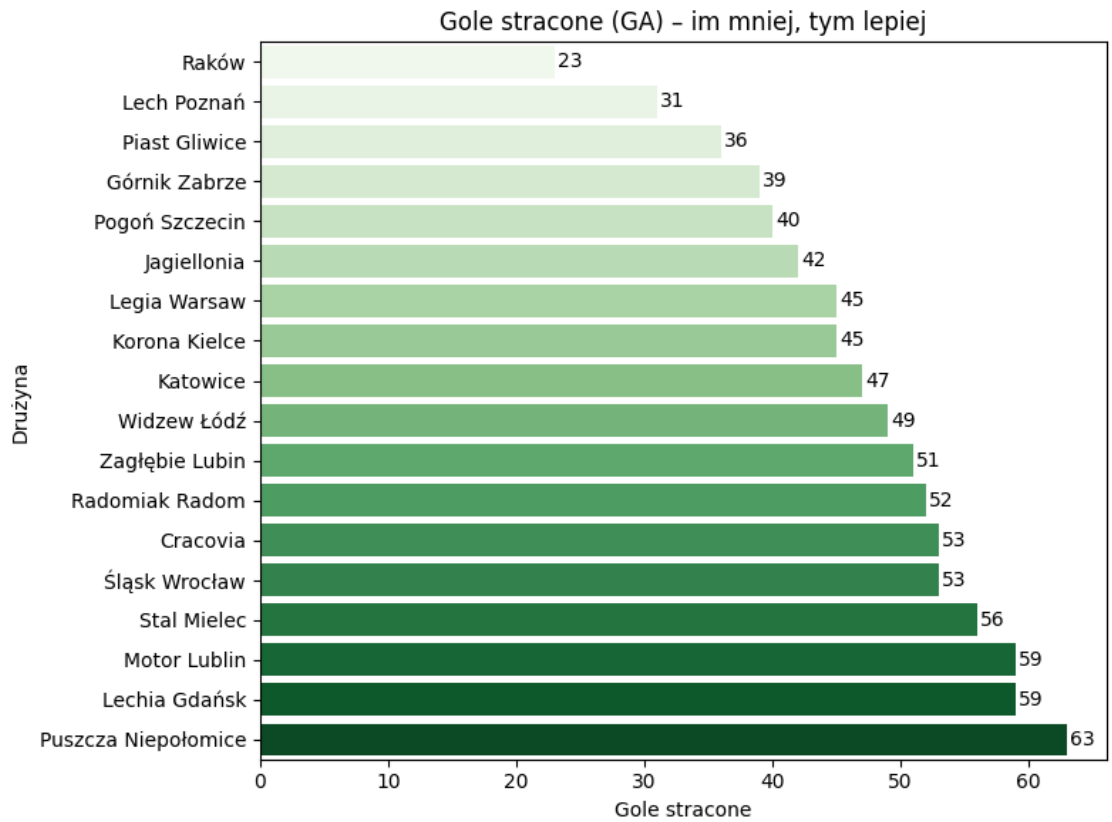
[74]:
```
df[['Squad','GA']].sort_values('GA', ascending=True)
```

[74]:

	Squad	GA
1	Raków	23
0	Lech Poznań	31
9	Piast Gliwice	36
8	Górnik Zabrze	39
3	Pogoń Szczecin	40
2	Jagiellonia	42
4	Legia Warsaw	45
10	Korona Kielce	45
7	Katowice	47
12	Widzew Łódź	49
14	Zagłębie Lubin	51
11	Radomiak Radom	52
5	Cracovia	53
16	Śląsk Wrocław	53
15	Stal Mielec	56

```
6          Motor Lublin  59
13        Lechia Gdańsk  59
17  Puszcza Niepołomice  63
```

[88]:
```
df_defense = df[['Squad', 'GA']].sort_values('GA', ascending=True)  # od␣
↪najlepszej obrony

plt.figure(figsize=(8,6))
sns.barplot(
    data=df_defense,
    x='GA',
    y='Squad',
    hue='Squad',
    palette='Greens',
    legend=False
)

plt.title("Gole stracone (GA) - im mniej, tym lepiej")
plt.xlabel("Gole stracone")
plt.ylabel("Drużyna")

for i, v in enumerate(df_defense['GA']):
    plt.text(v + 0.3, i, str(v), va='center')

plt.tight_layout()
plt.show()
```



3.4.3 Ranking strzałów

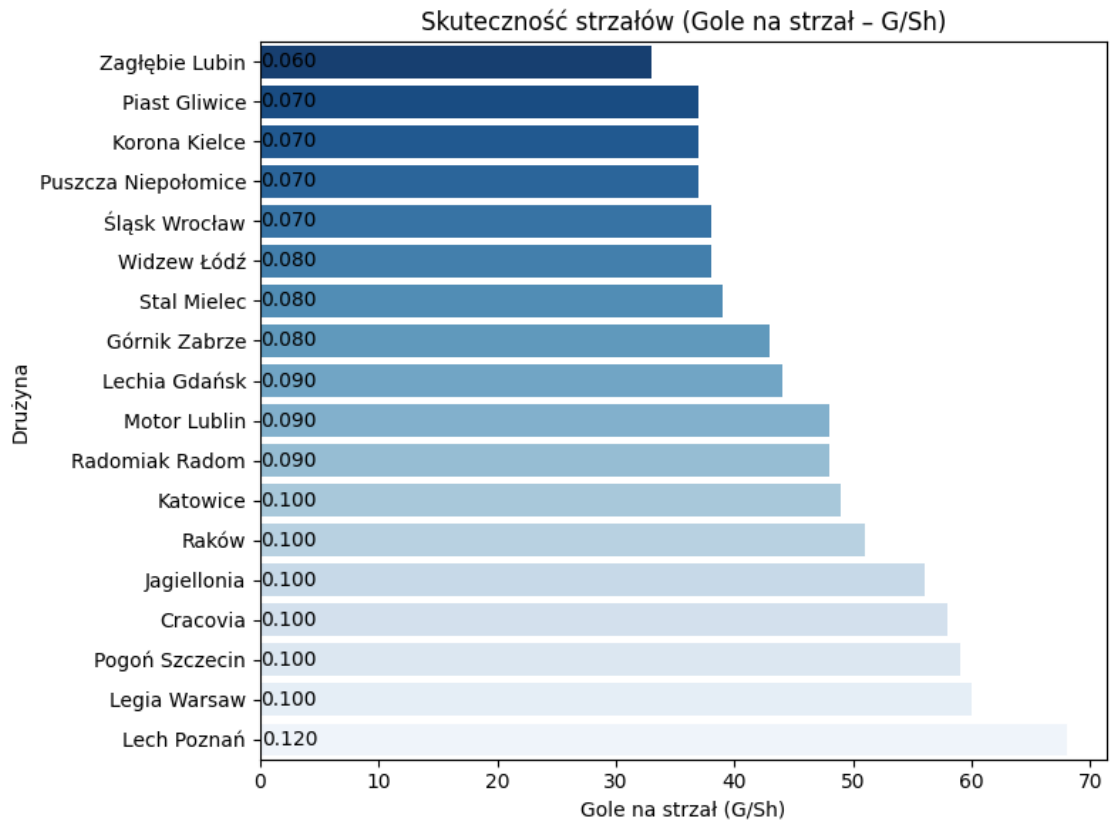
[77]:
```
df[['Squad','Sh/90','SoT/90']].sort_values('Sh/90', ascending=False)
```

[77]:

	Squad	Sh/90	SoT/90
0	Lech Poznań	15.62	5.38
4	Legia Warsaw	15.29	5.06
3	Pogoń Szczecin	15.09	5.29
8	Górnik Zabrze	14.76	4.82
7	Katowice	14.68	4.62
14	Zagłębie Lubin	14.47	4.38
16	Śląsk Wrocław	14.38	4.18
2	Jagiellonia	14.12	4.47
11	Radomiak Radom	14.00	4.21
5	Cracovia	13.85	4.29
1	Raków	13.79	4.82
15	Stal Mielec	13.50	4.18
9	Piast Gliwice	13.41	4.21
10	Korona Kielce	13.26	3.97
6	Motor Lublin	13.21	4.09

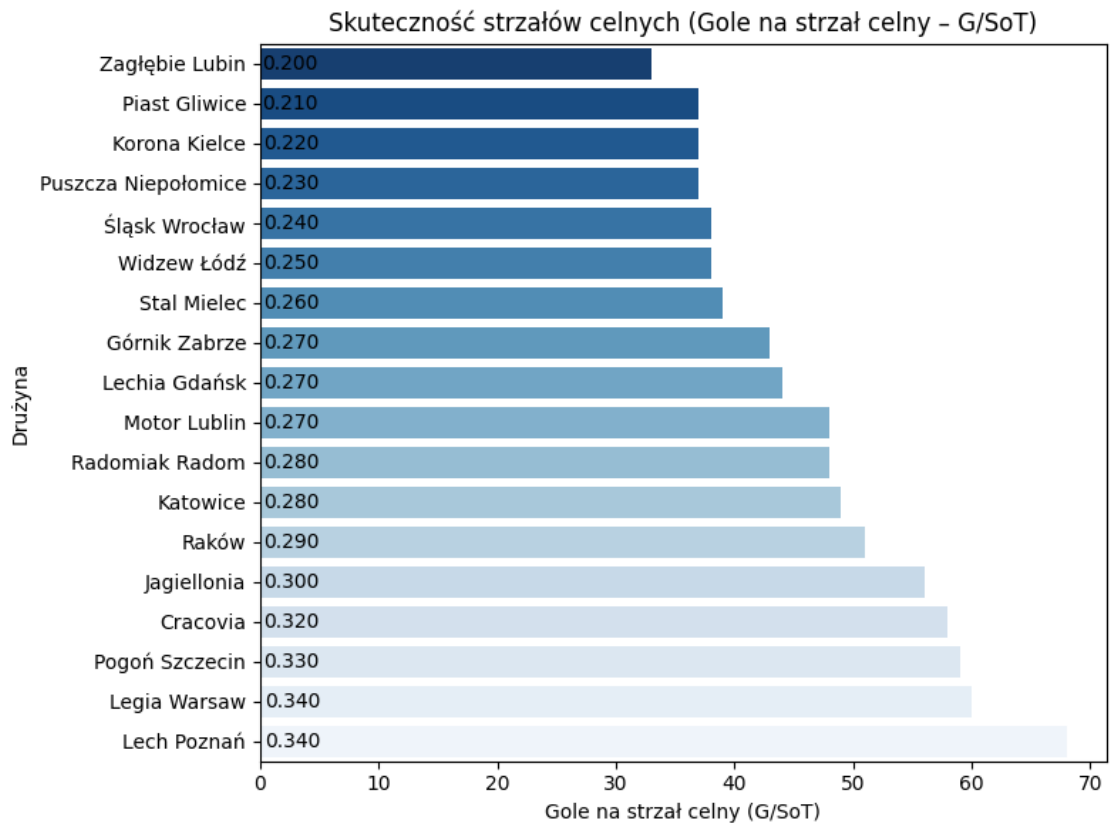
```
13        Lechia Gdańsk  12.53    4.18
12          Widzew Łódź  12.47    3.56
17  Puszcza Niepołomice  11.00    3.44
```

[89]:
```
df_eff = df[['Squad', 'G/Sh']].sort_values('G/Sh', ascending=True)

plt.figure(figsize=(8,6))
sns.barplot(
    data=df_offense,
    x='GF',
    y='Squad',
    hue='Squad',
    palette='Blues_r',
    legend=False
)


plt.title("Skuteczność strzałów (Gole na strzał - G/Sh)")
plt.xlabel("Gole na strzał (G/Sh)")
plt.ylabel("Drużyna")

for i, v in enumerate(df_eff['G/Sh']):
    plt.text(v + 0.001, i, f"{v:.3f}", va='center')

plt.tight_layout()
plt.show()
```



[93]:
```
df_eff_sot = df[['Squad', 'G/SoT']].sort_values('G/SoT', ascending=True)

plt.figure(figsize=(8,6))
sns.barplot(
    data=df_offense,
    x='GF',
    y='Squad',
    hue='Squad',
    palette='Blues_r',
    legend=False
)


plt.title("Skuteczność strzałów celnych (Gole na strzał celny - G/SoT)")
plt.xlabel("Gole na strzał celny (G/SoT)")
plt.ylabel("Drużyna")

for i, v in enumerate(df_eff_sot['G/SoT']):
    plt.text(v + 0.005, i, f"{v:.3f}", va='center')
```

```
plt.tight_layout()
plt.show()
```



3.4.4 Ranking posiadania

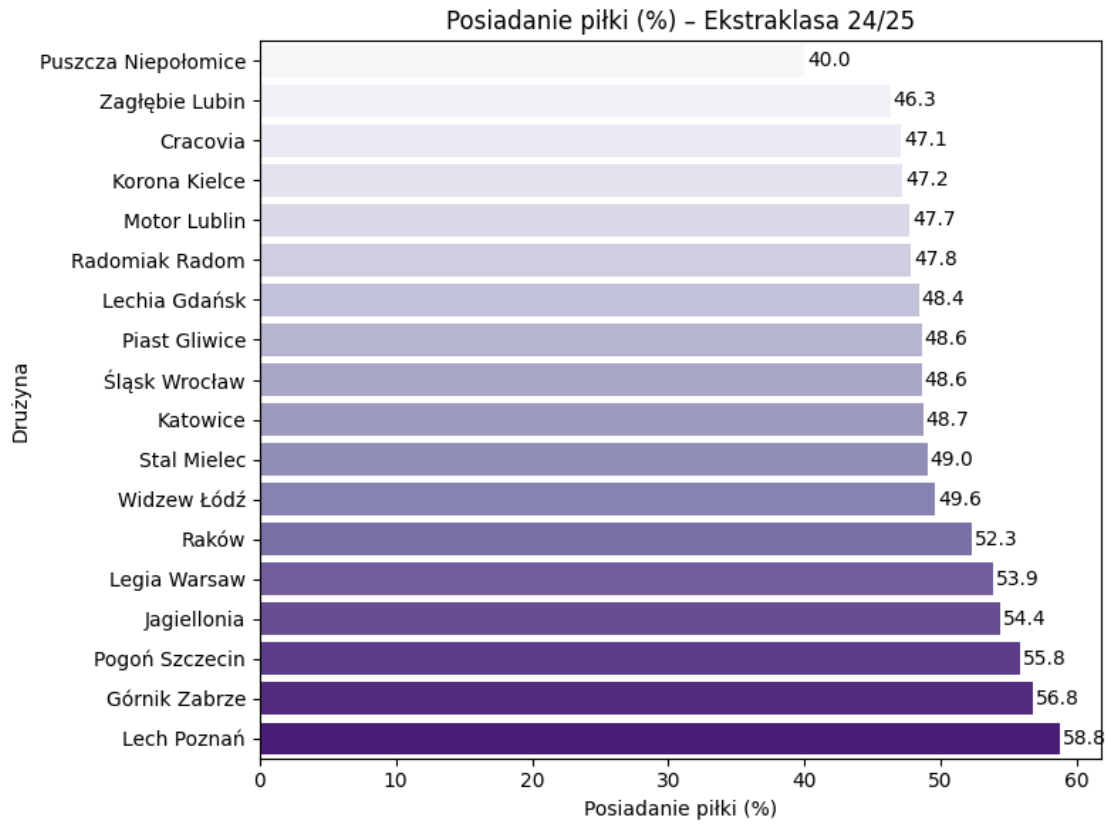
[76]:
```
df[['Squad','Poss']].sort_values('Poss', ascending=False)
```

[76]:

	Squad	Poss
0	Lech Poznań	58.8
8	Górnik Zabrze	56.8
3	Pogoń Szczecin	55.8
2	Jagiellonia	54.4
4	Legia Warsaw	53.9
1	Raków	52.3
12	Widzew Łódź	49.6
15	Stal Mielec	49.0
7	Katowice	48.7
9	Piast Gliwice	48.6
16	Śląsk Wrocław	48.6
13	Lechia Gdańsk	48.4

```
11       Radomiak Radom  47.8
6          Motor Lublin  47.7
10        Korona Kielce  47.2
5              Cracovia  47.1
14       Zagłębie Lubin  46.3
17  Puszcza Niepołomice  40.0
```

[90]:
```python
df_poss = df[['Squad', 'Poss']].sort_values('Poss', ascending=True)

plt.figure(figsize=(8,6))
sns.barplot(
    data=df_poss,
    x='Poss',
    y='Squad',
    hue='Squad',
    palette='Purples',
    legend=False
)

plt.title("Posiadanie piłki (%) - Ekstraklasa 24/25")
plt.xlabel("Posiadanie piłki (%)")
plt.ylabel("Drużyna")

for i, v in enumerate(df_poss['Poss']):
    plt.text(v + 0.2, i, f"{v:.1f}", va='center')

plt.tight_layout()
plt.show()
```



3.5 Porównanie grup top4, Mid, Bottom 3

[79]:
```
# Dodanie podziału na grupy (Top 4, Mid, Bottom 3)
def tier(rk):
    if rk <= 4:
        return "Top 4"
    elif rk >= 16:
        return "Bottom 3"
    else:
        return "Mid"

df['Tier'] = df['Rk'].apply(tier)
df['Tier'].value_counts()
```

[79]:
```
Tier
Mid         11
Top 4        4
Bottom 3     3
Name: count, dtype: int64
```

3.5.1 Porównanie statystyk między grupami

[ ]:
```
metrics = [
    'Pts', 'GF', 'GA', 'GD', 'Poss',
    'Sh/90', 'SoT/90', 'G/Sh', 'G/SoT'
]

df.groupby('Tier')[metrics].mean().round(2)
```

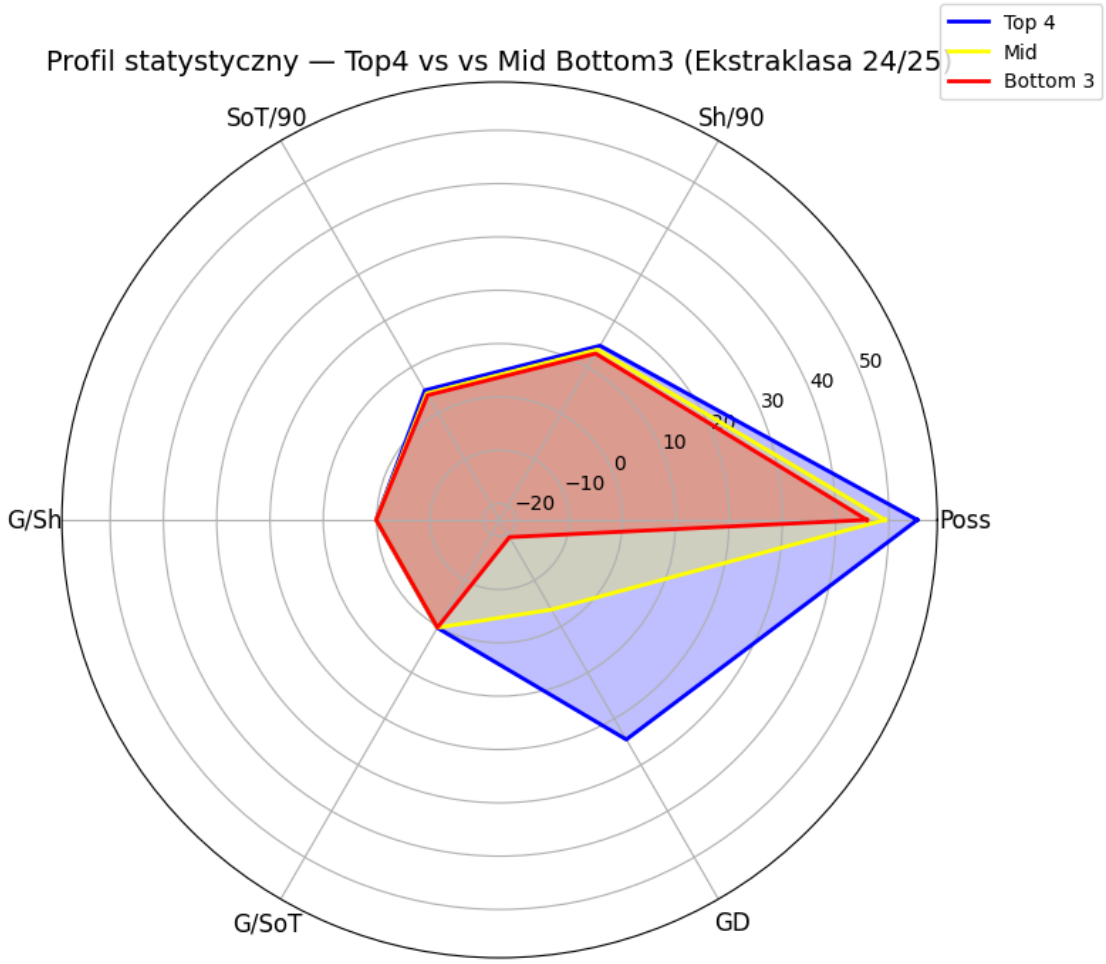
```
            Pts    GF     GA     GD   Poss  Sh/90  SoT/90  G/Sh  G/SoT
Tier
Bottom 3  29.67  38.0  57.33 -19.33  45.87  12.96    3.93  0.07   0.24
Mid       44.91  45.0  48.64  -3.64  49.28  13.81    4.31  0.08   0.27
Top 4     64.50  58.5  34.00  24.50  55.32  14.66    4.99  0.11   0.31
```

Statystyki opisowe wyglądają zgodnie z logiką, czyli im wyższa grupa w tabeli tym lepsza średnia z wyników poszczególnych zmiennych.

3.5.2 Radar chart top4 vs mid vs bottom 3

[83]:
```
import numpy as np
import matplotlib.pyplot as plt

# Średnie dla grup
top4 = df[df['Tier'] == 'Top 4'][['Poss','Sh/90','SoT/90','G/Sh','G/SoT','GD']].
 ↪mean()
Mid = df[df['Tier'] == 'Mid'][['Poss','Sh/90','SoT/90','G/Sh','G/SoT','GD']].
 ↪mean()
bottom3 = df[df['Tier'] == 'Bottom 3'][['Poss','Sh/90','SoT/90','G/Sh','G/
 ↪SoT','GD']].mean()

# Kategorie
categories = ['Poss', 'Sh/90', 'SoT/90', 'G/Sh', 'G/SoT', 'GD']
N = len(categories)

# Konwersja do list + zamknięcie wykresu (pierwsza wartość = ostatnia)
top4_vals = top4.tolist()
Mid_vals = Mid.tolist()
bottom3_vals = bottom3.tolist()

top4_vals += top4_vals[:1]
Mid_vals += Mid_vals[:1]
bottom3_vals += bottom3_vals[:1]

# Kąty
angles = np.linspace(0, 2 * np.pi, N, endpoint=False).tolist()
angles += angles[:1]
```

```python
# Wykres
plt.figure(figsize=(8,8))
plt.subplot(111, polar=True)

# Top4
plt.plot(angles, top4_vals, label='Top 4', linewidth=2, color='blue')
plt.fill(angles, top4_vals, alpha=0.25, color='blue')

#Mid
plt.plot(angles, Mid_vals, label='Mid', linewidth=2, color='yellow')
plt.fill(angles, Mid_vals, alpha=0.25, color='yellow')

# Bottom3
plt.plot(angles, bottom3_vals, label='Bottom 3', linewidth=2, color='red')
plt.fill(angles, bottom3_vals, alpha=0.25, color='red')

# Ustawienia
plt.xticks(angles[:-1], categories, fontsize=12)
plt.title("Profil statystyczny - Top4 vs vs Mid Bottom3 (Ekstraklasa 24/25)",␣
 ↪fontsize=14)
plt.legend(loc='upper right', bbox_to_anchor=(1.2, 1.1))
plt.show()
```



Zespoły z czołówki cechują się zdecydowanie większą intensywnością ataku: częściej utrzymują się przy piłce, dłużej kontrolują przebieg meczu i regularnie znajdują się na połowie przeciwnika. Drużyny ze strefy spadkowej odstają nie tylko pod względem liczby kreowanych okazji, ale również pod względem stabilności defensywnej. Strata dużej liczby bramek oraz częste błędy w obronie sprawiają, że zespoły te rzadko mają możliwość narzucenia własnego stylu gry, a wiele ich akcji kończy się zanim zdążą stworzyć realne zagrożenie. Dlatego widać zależność co do miejsca w tabeli w przypadku posiadania piłki i różnicy bramek.

3.6 Charakterystyki drużyn: Lech Poznań vs Widzew Łódź

[ ]:
```
lech = df[df['Squad'] == 'Lech Poznań']
widzew = df[df['Squad'] == 'Widzew Łódź']

stats = ['Poss','Sh/90','SoT/90','G/Sh','G/SoT','GF','GA','GD','Pts']

comparison = pd.DataFrame({
    'Statystyka': stats,
```

```
    'Lech Poznań': lech[stats].iloc[0].values,
    'Widzew Łódź': widzew[stats].iloc[0].values
})

comparison
```

[ ]:

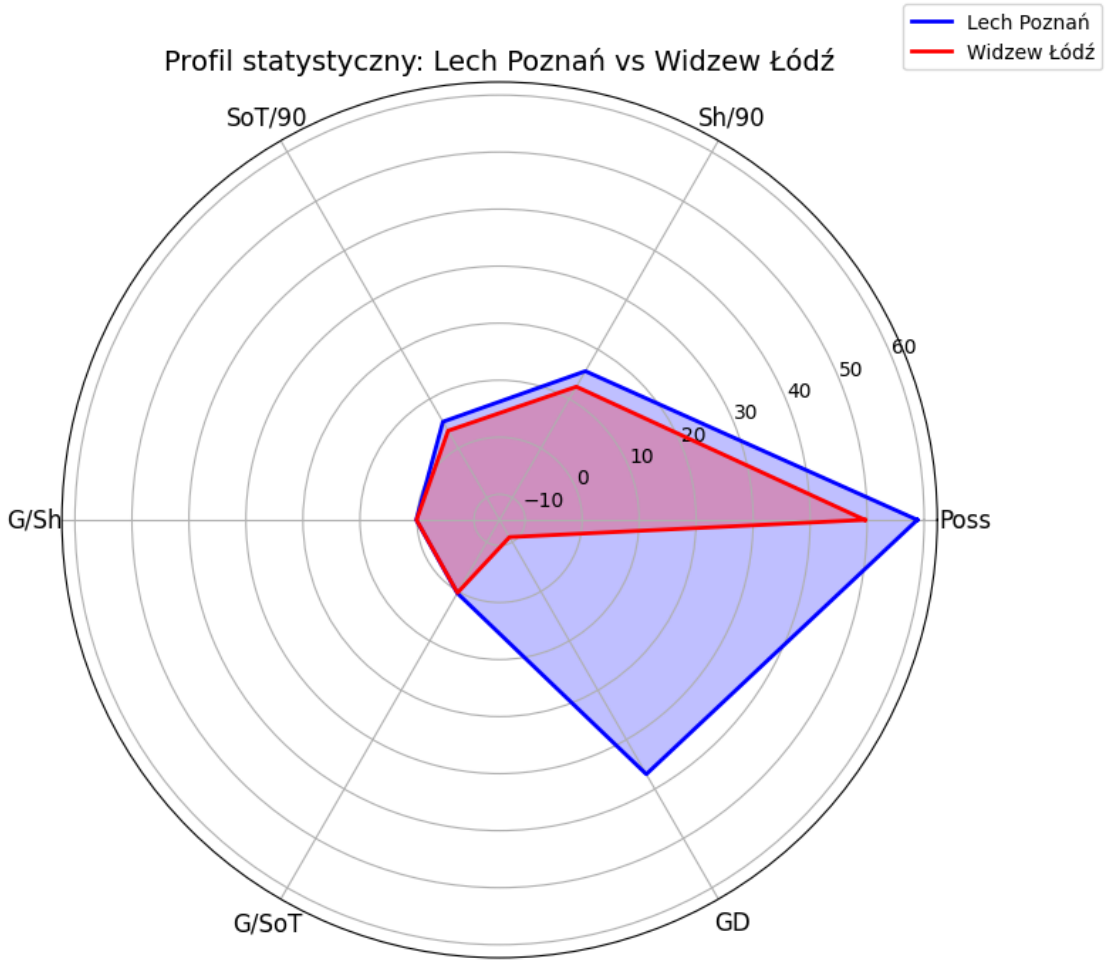
	Statystyka	Lech Poznań	Widzew Łódź
0	Poss	58.80	49.60
1	Sh/90	15.62	12.47
2	SoT/90	5.38	3.56
3	G/Sh	0.12	0.08
4	G/SoT	0.34	0.27
5	GF	68.00	38.00
6	GA	31.00	49.00
7	GD	37.00	-11.00
8	Pts	70.00	40.00

3.6.1 Wykres RADAR (Lech vs Widzew)

[86]:

```
import numpy as np
import matplotlib.pyplot as plt

# wartości
lech_vals = lech[['Poss','Sh/90','SoT/90','G/Sh','G/SoT','GD']].iloc[0].values.
 ↪tolist()
widzew_vals = widzew[['Poss','Sh/90','SoT/90','G/Sh','G/SoT','GD']].iloc[0].
 ↪values.tolist()

categories = ['Poss','Sh/90','SoT/90','G/Sh','G/SoT','GD']
N = len(categories)

# domknięcie wykresu
lech_plot = lech_vals + lech_vals[:1]
widzew_plot = widzew_vals + widzew_vals[:1]

angles = np.linspace(0, 2*np.pi, N, endpoint=False).tolist()
angles += angles[:1]

plt.figure(figsize=(8,8))
plt.subplot(111, polar=True)

# Lech
plt.plot(angles, lech_plot, label='Lech Poznań', linewidth=2, color='blue')
plt.fill(angles, lech_plot, alpha=0.25, color='blue')

# Widzew
plt.plot(angles, widzew_plot, label='Widzew Łódź', linewidth=2, color='red')
```

```
plt.fill(angles, widzew_plot, alpha=0.25, color='red')

plt.xticks(angles[:-1], categories, fontsize=12)
plt.title("Profil statystyczny: Lech Poznań vs Widzew Łódź", fontsize=14)
plt.legend(loc='upper right', bbox_to_anchor=(1.2, 1.1))

plt.show()
```



Lech prezentuje profil drużyny dominującej zarówno pod względem stylu gry, jak i efektywności. Utrzymywał jedno z najwyższych w lidze posiadanie piłki, co przekładało się na większą liczbę akcji ofensywnych, dodatnią różnicę barmek oraz znacznie wyższy wolumen strzałów i strzałów celnych. Drużyna Widzewa natomiast generowała mniejszą liczbę strzałów, rzadziej dochodziła do czystych pozycji strzeleckich, miała ujemny bilans bramkowy, a jej posiadanie piłki oscylowało wokół wartości typowych dla zespołów środka tabeli. Studium przypadku jasno pokazuje, że różnica punktowa pomiędzy tymi zespołami nie wynika z jednego, pojedynczego wskaźnika. Jest efektem skumulowanego wpływu wielu elementów, w których Lech konsekwentnie przewyższał swojego rywala.

3.7 Porównanie grup top4, Mid, Bottom 3

[ ]:
```
# Dodanie podziału na grupy (Top 4, Mid, Bottom 3)
def tier(rk):
    if rk <= 4:
        return "Top 4"
    elif rk >= 16:
        return "Bottom 3"
    else:
        return "Mid"

df['Tier'] = df['Rk'].apply(tier)
df['Tier'].value_counts()
```

```
Tier
Mid         11
Top 4        4
Bottom 3     3
Name: count, dtype: int64
```

3.7.1 Porównanie statystyk między grupami

[ ]:
```
metrics = [
    'Pts', 'GF', 'GA', 'GD', 'Poss',
    'Sh/90', 'SoT/90', 'G/Sh', 'G/SoT'
]

df.groupby('Tier')[metrics].mean().round(2)
```

	Pts	GF	GA	GD	Poss	Sh/90	SoT/90	G/Sh	G/SoT
Tier									
Bottom 3	29.67	38.0	57.33	-19.33	45.87	12.96	3.93	0.07	0.24
Mid	44.91	45.0	48.64	-3.64	49.28	13.81	4.31	0.08	0.27
Top 4	64.50	58.5	34.00	24.50	55.32	14.66	4.99	0.11	0.31

Statystyki opisowe wyglądają zgodnie z logiką, czyli im wyższa grupa w tabeli tym lepsza średnia z wyników poszczególnych zmiennych.

4 Predictive Modeling

[ ]:
```
features = [
    'GF',       # gole strzelone
    'GA',       # gole stracone
    'Poss',     # posiadanie piłki
    'Sh/90',    # strzały na 90
    'SoT/90',   # strzały celne na 90
    'G/Sh',     # skuteczność strzałów
    'G/SoT'     # skuteczność celnych
```

```
]

X = df[features]
y = df['Pts']
```

[ ]:
```
# Standaryzacja
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

4.1 model regresji liniowej

[ ]:
```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_scaled, y)

model.coef_, model.intercept_
```

```
(array([-0.86930394, -7.63277263, -1.31613059, -0.66830161,  3.43953687,
         1.57335207,  5.56108167]),
 np.float64(46.72222222222222))
```

[ ]:
```
importance = pd.DataFrame({
    'Feature': features,
    'Coefficient': model.coef_
}).sort_values('Coefficient', ascending=False)

importance
```

```
  Feature  Coefficient
6   G/SoT     5.561082
4  SoT/90     3.439537
5    G/Sh     1.573352
3   Sh/90    -0.668302
0      GF    -0.869304
2    Poss    -1.316131
1      GA    -7.632773
```

Jeśli skuteczność strzałów celnych wzrośnie o 1, to liczba punktów wzrośnie o 5,56 punktu. Jeśli drużyna oddaje o 1 strzał celny więcej na mecz, to liczba punktów wzrasta o 3,44 punktu w sezonie. Jeśli skuteczność strzałów wzrośnie o 1 jednostkę (100 p.p.), liczba punktów wzrośnie o 1,57 punktu. Jeśli drużyna oddaje o 1 strzał (niekoniecznie celny) więcej na mecz, liczba punktów spada o 0,67 punktu. Jeśli posiadanie piłki wzrośnie o 1 punkt procentowy, liczba punktów spada o 1,32 punktu, to sugeruje, że w tej próbie drużyny z większym posiadaniem nie zdobywają więcej punktów. Jeśli drużyna strzeli 1 gola więcej w sezonie, liczba punktów spada o 0,87 punktu, co jest nielogiczne sportowo. Jeśli drużyna straci o 1 bramkę więcej w sezonie, liczba punktów spada o 7,63 punktu i jest to najsilniejszy wpływ w modelu.

4.1.1 R^2

[ ]:
```
from sklearn.metrics import r2_score

y_pred = model.predict(X_scaled)
print("R²:")
r2_score(y, y_pred)
```

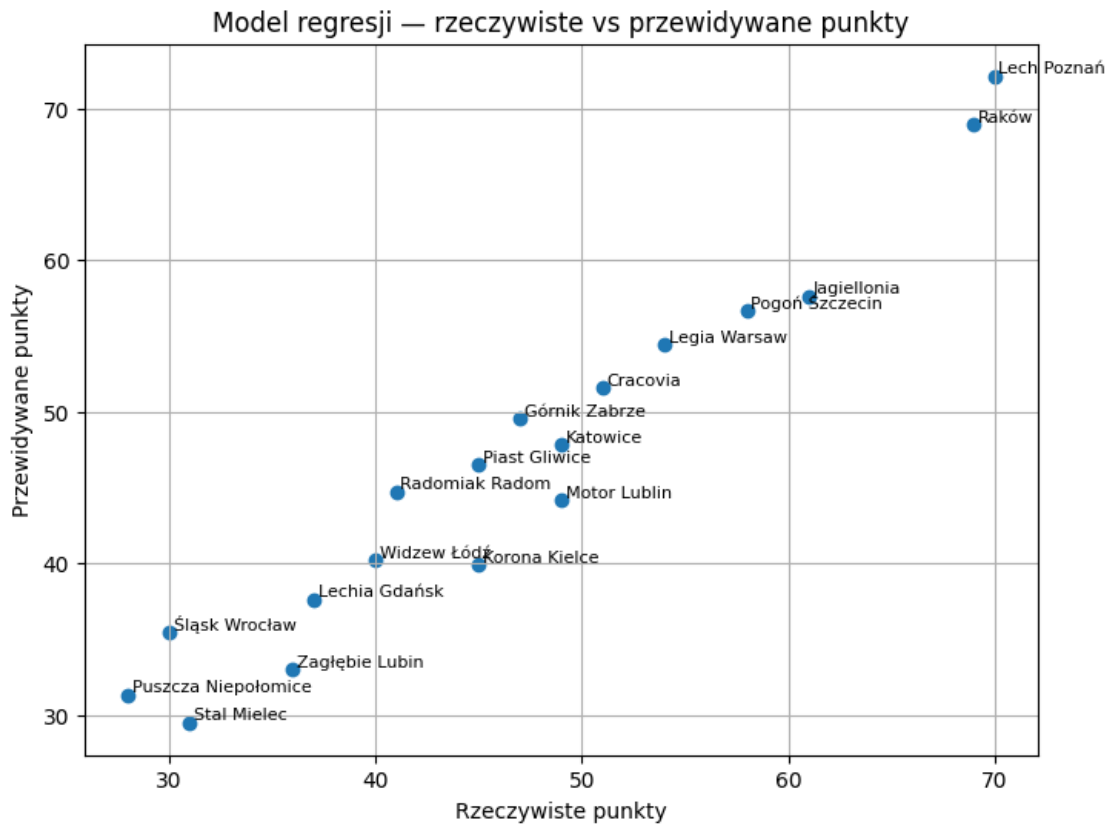
```
R²:
```

```
0.9451880698542499
```

94.5% zmienności liczby punktów drużyn w lidze jest wyjaśnione przez zmienne w modelu. Jest to bardzo dobry wynik.

4.2 Przewidywane punkty vs rzeczywiste

[ ]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))
plt.scatter(y, y_pred)

for i, row in df.iterrows():
    plt.text(row['Pts'] + 0.2, y_pred[i] + 0.2, row['Squad'], fontsize=8)

plt.xlabel("Rzeczywiste punkty")
plt.ylabel("Przewidywane punkty")
plt.title("Model regresji - rzeczywiste vs przewidywane punkty")
plt.grid(True)
plt.show()
```



[105]:
```
df['Diff'] = df['Pts'] - df['Predicted_Pts']

df[['Squad', 'Pts', 'Predicted_Pts', 'Diff']].sort_values('Diff',␣
↪ascending=False)
```

[105]:

	Squad	Pts	Predicted_Pts	Diff
10	Korona Kielce	45	39.920302	5.079698
6	Motor Lublin	49	44.165639	4.834361
2	Jagiellonia	61	57.608251	3.391749
14	Zagłębie Lubin	36	32.998360	3.001640
15	Stal Mielec	31	29.496075	1.503925
3	Pogoń Szczecin	58	56.677336	1.322664
7	Katowice	49	47.797315	1.202685
1	Raków	69	68.905237	0.094763
12	Widzew Łódź	40	40.189111	-0.189111
4	Legia Warsaw	54	54.386850	-0.386850
5	Cracovia	51	51.606787	-0.606787
13	Lechia Gdańsk	37	37.625374	-0.625374
9	Piast Gliwice	45	46.479511	-1.479511
0	Lech Poznań	70	72.123751	-2.123751

```
8          Górnik Zabrze   47      49.530769 -2.530769
17  Puszcza Niepołomice   28      31.342878 -3.342878
11        Radomiak Radom   41      44.710584 -3.710584
16         Śląsk Wrocław   30      35.435871 -5.435871
```

Model umiarkowanie poradził sobie z przewidywaniem ilości punktów zdobytych przez drużyny. Natomiast pokazuje to że według modelu są drużyny które uzyskały wyniki powyżej (Korona, Motor, Jagielonia) lub poniżej (Puszcza, Radomiak, Śląsk) swojego potencjału.

5 Key Findings

Najważniejszym z wniosków jest dominująca rola efektywności ofensywnej. Drużyny osiągające wysokie miejsca w tabeli charakteryzowały się nie tylko większą liczbą oddawanych strzałów, ale przede wszystkim wyższą jakością i skutecznością tych prób. Drugim kluczowym wnioskiem jest wyraźna rola organizacji gry defensywnej, czego najlepszym przykładem pozostaje Raków. Jego wysoka pozycja w tabeli była wynikiem przede wszystkim wzorowo funkcjonującej defensywy, która redukowała liczbę strat bramkowych i pozwalała na zdobywanie punktów nawet w meczach o ograniczonej liczbie sytuacji ofensywnych. Model regresji potwierdził wagę tego czynnika, wskazując liczbę straconych bramek jako jedną z najsilniej, negatywnie skorelowanych zmiennych w odniesieniu do punktów. Wreszcie analiza expected points pokazała, że część drużyn uzyskiwała wyniki powyżej lub poniżej potencjału wynikającego ze statystyk.

6 Recommendations

Analiza wskazuje, że drużyny z dolnej części tabeli Ekstraklasy powinny przede wszystkim poprawić liczbę tworzonych sytuacji ofensywnych, ponieważ generują zbyt mało okazji do zdobycia bramki. Drugim istotnym obszarem jest organizacja defensywy, ponieważ liczba straconych bramek silnie wpływa na zdobycz punktową drużyn. Poprawa dyscypliny taktycznej, ustawienia w polu karnym oraz zachowania w fazach przejściowych może ograniczyć straty wynikające z błędów indywidualnych i braku asekuracji. Dodatkowo większe wykorzystanie analityki sportowej, w tym analiz expected points, może pomóc drużynom lepiej wykorzystywać swój potencjał i podejmować trafniejsze decyzje taktyczne oraz kadrowe.